

GSO-SLAM: Bidirectionally Coupled Gaussian Splatting and Direct Visual Odometry

2026 (RA-L)

Jiung Yeon, Seongbo Ha, Hyeonwoo Yu†

Sungkyunkwan University

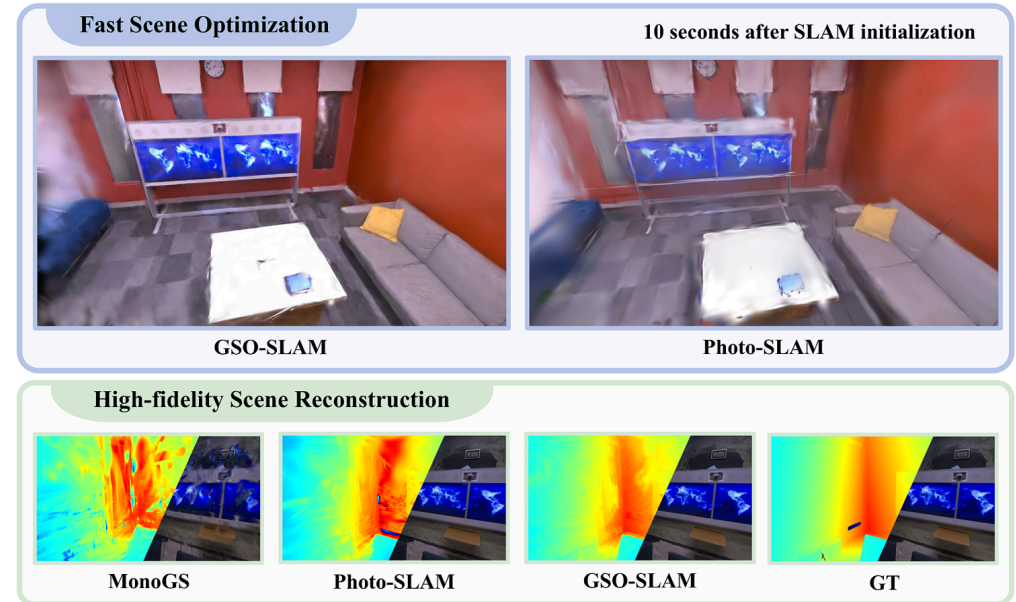
2026.02.27

Contents

1. Introduction
2. Method
3. Experiments
4. Conclusion

Introduction

- SLAM(Simultaneous Localization and Mapping)은 로봇, AR/VR 등에서 필수적인 기술
 - 정확한 pose estimation뿐만 아니라, 기하학적으로 정밀한 dense reconstruction에 대한 수요 증가
- INR(Implicit Neural Representation) 기반 SLAM이 등장했으나, 높은 연산 비용으로 실시간 적용에 한계
 - iMAP, NICE-SLAM, Co-SLAM, Point-SLAM 등
- Gaussian Splatting(GS) 기반 SLAM이 대안으로 부상
 - 명시적 Gaussian primitive와 GPU 가속 rasterization 기반 렌더링
 - MonoGS, SplaTAM, GS-SLAM 등 → GS의 dense SLAM에서의 효과성 입증



Introduction

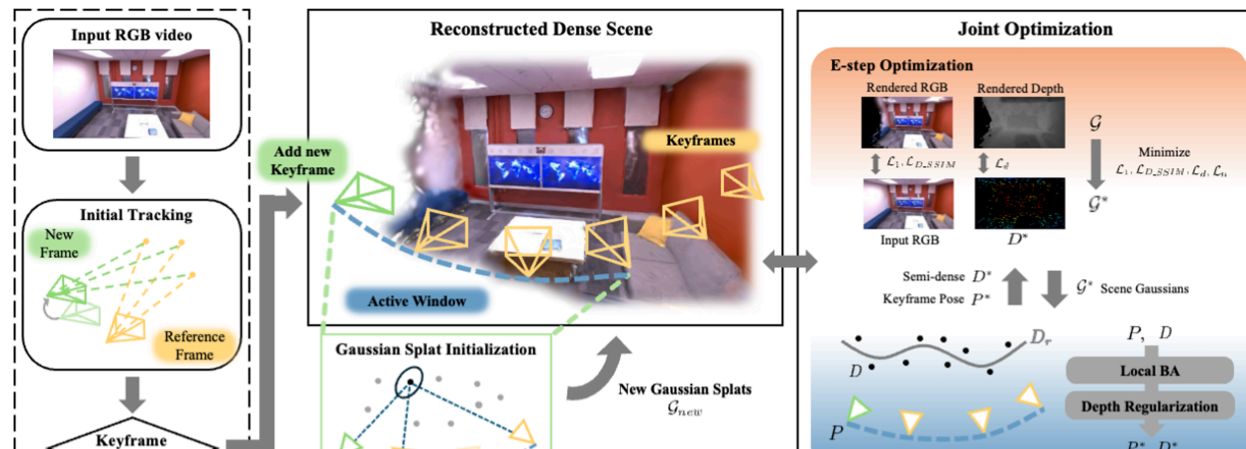
Limitations of Existing GS-SLAM Methods

- **Coupled 방식** (MonoGS, SplatAM, GS-SLAM):
 - 통합된 scene을 tracking과 mapping에 공유하여 re-rendering loss 기반 tracking 수행
 - 하지만 tracking에 반복적인 rendering과 optimization이 필요 → 실시간 처리 어려움
- **Loosely coupled 방식** (Photo-SLAM, MGSO):
 - ORB-SLAM3, DSO 등 기존 tracking framework와 GS를 독립적으로 통합
 - 속도 문제는 해결하지만, tracking과 mapping이 독립적으로 동작 → dense scene을 tracking에 활용 불가
 - Scene geometry 정보가 scene optimization에 제한적으로만 반영

Introduction

GSO-SLAM의 제안

- **Bidirectional Coupling:** Visual Odometry(VO)와 Gaussian Splatting(GS)을 양방향으로 결합
 - DSO(Direct Sparse Odometry)와 2D Gaussian Splatting 기반 mapping의 joint optimization
- **EM(Expectation-Maximization) Framework:**
 - Joint optimization을 EM으로 정식화 → 추가 연산 비용 없이 VO의 semi-dense depth와 GS representation을 동시에 최적화
- **Gaussian Splat Initialization:**
 - DSO의 image gradient, keyframe pose, pixel association을 활용하여 초기 Gaussian 파라미터를 결정
 - Heuristic 초기화 방법 불필요 → 수렴 가속화



Method

Preliminaries - Direct Sparse Odometry (DSO)

- Direct Sparse Odometry (DSO)는, 이미지를 사용해서 loss를 구하는 photometric loss기반이라 'Direct'라 씀
- 하지만, 모든 이미지 픽셀을 사용하는것은 쉽지 않다.(연산량 너무많음)
- 따라서, 그 중 gradient가 큰 픽셀만 선택해서 사용하기에 'Sparse'라고 함
 - 그래서 'Direct Sparse Odometry'라고 불리는 것
- DSO는, 이미지 I 로부터, 최적의 camera poses P^* 와 depth maps D^* 를 추정하는 문제로 정의
 - 관측된 이미지 I 를 가장 잘 설명하는 P^* 와 D^* 를 찾자..!

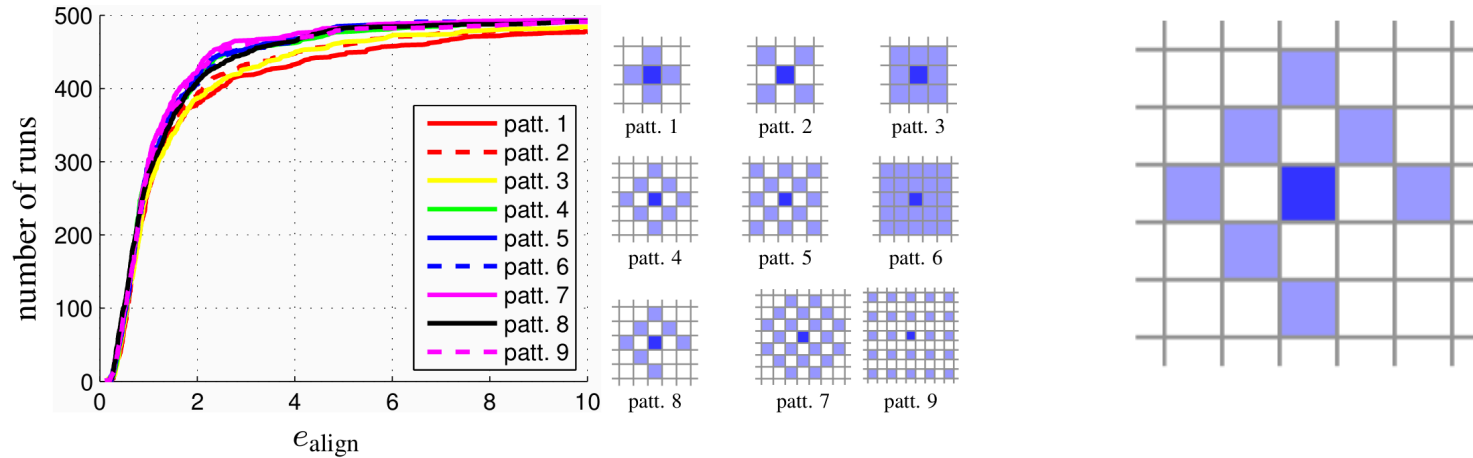
$$P^*, D^* = \arg \max_{P, D} p(I \mid P, D)$$

- 어떻게..?
 - Photometric loss를 구해서, 오차를 최소화하는 방식

Method

$$E_{pj} = \sum_{\mathbf{p} \in \mathcal{N}_p} \left\| (I_j[\mathbf{p}'] - b_j) - \frac{t_j e^{a_j}}{t_i e^{a_i}} (I_i[\mathbf{p}] - b_i) \right\|$$

- 여기서 E_{pj} 는, reference frame i 의 pixel $\mathbf{p}$ 와 target frame j 의 reprojected pixel $\mathbf{p}'$ 간의 photometric error
- Sparse라며,, $\mathbf{p}$ 는 어떻게 찾는가?
 - 32x32 크기의 block으로 이미지를 나누고, 각 pixel gradient의 절댓값중 중앙값에서 + 7 threshold로 설정
 - 이렇게 하면, 하얀 벽처럼 textureless한 곳에서도 픽셀을 뽑을 수 있음(well-distributed)
 - 선택된 pixel $\mathbf{p}$ 는, 특정 pattern을 만들어, 이들은 $\mathcal{N}_p$ 에 속함



Method

$$E_{pj} = \sum_{\mathbf{p} \in \mathcal{N}_p} \left\| (I_j[\mathbf{p}'] - b_j) - \frac{t_j e^{a_j}}{t_i e^{a_i}} (I_i[\mathbf{p}] - b_i) \right\|$$

- $\mathbf{p}'$ 은, 아래와 같이 reprojection으로 구할 수 있음

$$\mathbf{p}' = \Pi_c (\mathbf{R} \Pi_c^{-1}(\mathbf{p}, d_p) + \mathbf{t})$$

- 여기서, d_p 는 pixel의 inverse depth이고, reference frame에서의 pixel을 unprojection하고, 좌표계 변환을 거친 뒤 re-projection을 통해 current frame에서의 pixel $\mathbf{p}'$ 를 구할 수 있음
- 여기서, 조도의 변화를 보정하기 위해 bias를 빼고, exposure time을 보정하는 항이 추가됨
- 마지막으로 앞서 구한 photometric error를 모든 keyframe과 선정된 pixel들에 대해 합산하면 photometric loss가 됨

$$E_{\text{photo}} := \sum_{i \in \mathcal{F}} \sum_{\mathbf{p} \in \mathcal{P}_i} \sum_{j \in \text{obs}(\mathbf{p})} E_{\mathbf{p}j}$$

Method

Preliminaries - 2D Gaussian Splatting (2DGS)

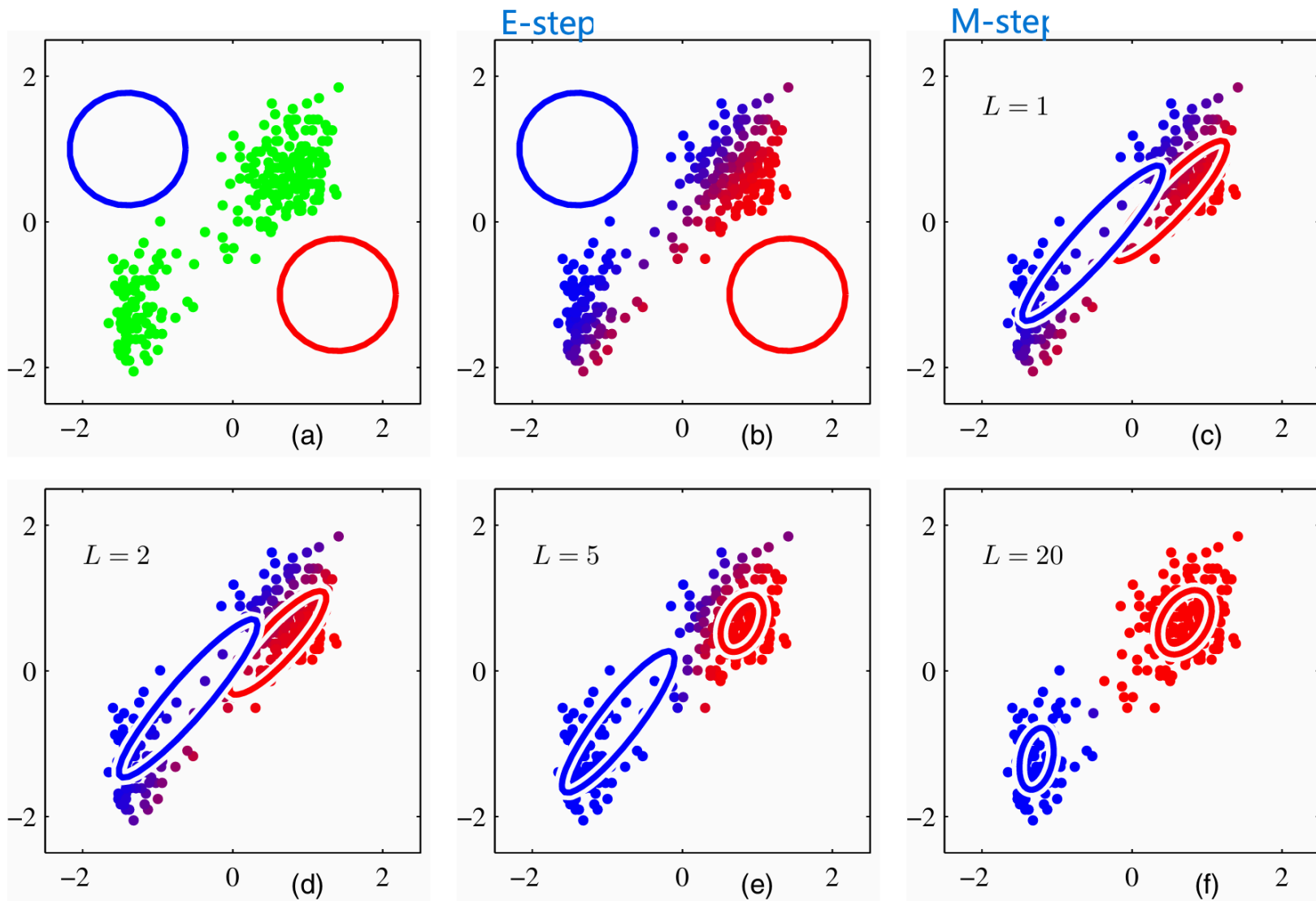
- 3D가 아닌 **2DGS**를 썼다 왜?
- 이미지만으로 dense 3D geometry를 추정하는 것은 정보가 부족(2D로부터 3D)한 까다로운 문제(ill-posed)이므로, 이를 효과적으로 모델링하기 위해 2D Gaussian Splatting 사용
- 각 2D 가우시안은 (μ, Σ, α, c) 으로 정의되며, 렌더링과 최적화 과정의 연산 부담을 줄이기 위해 복잡한 Spherical Harmonics(SH) 제외(RGB color만 사용?)
- 3D 공간의 교차점을 평가하는 3DGS와 달리, 2DGS는 **ray와 2DGS의 교차(ray-splat intersection)** 방식을 사용하여 기하학적 모호함을 줄임.
- 이러한 2DGS의 고유한 렌더링 방식 덕분에 **다양한 시점에서 봐도 일관된 깊이(multi-view consistent depth)**를 얻을 수 있어, SLAM이나 표면 재구성(surface reconstruction) 작업에서 기하학적 정확도가 훨씬 뛰어남.

Method

Joint optimization

- 단순히 Bundle Adjustment(BA)를 통해 얻은 pose와 point로 2DGS에게 넘겨줘서 최적화 하는 방식은, photometric loss로 2DGS만 최적화되는데, pose가 정확하지 않을 때에는 error을 가진채로 열심히 최적화
 - 이상한 곳으로 빠져 열심히 이상한 일을 한다.
- 따라서, 본 논문에서는 **Joint optimization**을 통해 BA와 2DGS가 서로 영향을 주고받으며 최적화 되도록 제안
 - DSO tracking과 dense reconstruction이 상호작용하여, 통합된 프레임워크 내에서 error를 공동으로 수정
 - SLAM에선 Joint optimization, tightly coupled 방식이라고 하기도 하지요..?

Method



Method

EM Algorithm의 GSO-SLAM으로의 적용

- PRML의 GMM에서 봤던 EM과 동일한 구조 $\rightarrow$ GSO-SLAM에 그대로 적용

PRML (GMM)	GSO-SLAM
Observed data $\mathbf{X}$	이미지 시퀀스 I
Latent variable $\mathbf{Z}$	2D Gaussian scene $\mathcal{G}$
Parameters $\theta = (\pi, \mu, \Sigma)$	Camera pose & depth (P, D)
E-step: responsibility $\gamma(z_{nk})$ 계산	E-step: $\mathcal{G}^*$ MAP 추정 (scene 업데이트)
M-step: μ_k, Σ_k, π_k 업데이트	M-step: (P, D) 업데이트 (BA + Depth Reg.)

Method

Joint Optimization: E-step

- P 와 D 를 고정, $\mathcal{G}$ 를 MAP로 업데이트:

$$\mathcal{L}(q, P, D) = \mathbb{E}_{q(\mathcal{G})}[\log p(I, D, \mathcal{G}|P)] \quad (5)$$

- $q(\mathcal{G}) \approx \delta(\mathcal{G} - \mathcal{G}^*)$ (δ -function 근사) $\rightarrow$ MAP estimation으로 단순화:

$$\begin{aligned} \mathcal{G}^* &= \arg \max_{\mathcal{G}} \log p(\mathcal{G}|I, D, P) \propto \arg \max_{\mathcal{G}} \log p(I|\mathcal{G}, D, P) p(D|\mathcal{G}, P) p(\mathcal{G}|P) \\ &= \arg \min_{\mathcal{G}} \underbrace{-\log p(I|\mathcal{G}, D, P)}_{\text{RGB Rendering Loss}} \underbrace{-\log p(D|\mathcal{G}, P)}_{\text{Semi-dense Depth Loss}} \underbrace{-\log p(\mathcal{G}|P)}_{\text{Normal Consistency Loss}} \end{aligned} \quad (6)$$

- 각 loss를 구체화하면:

$$\mathcal{G}^* = \arg \min_{\mathcal{G}} \underbrace{(1 - \lambda)\mathcal{L}_1(I_r, I_{gt}) + \lambda \mathcal{L}_{D\text{-SSIM}}(I_r, I_{gt})}_{\text{RGB}} + \underbrace{\lambda_d \mathcal{L}_1(D_r, D)}_{\text{Depth}} + \underbrace{\lambda_n \mathcal{L}_n}_{\text{Normal}} \quad (7)$$

Method

Joint Optimization: M-step

- E-step에서 얻은 $\mathcal{G}^*$ 를 고정, P 와 D 를 업데이트:

$$\begin{aligned} P^*, D^* &= \arg \max_{P, D} \log p(I, D, \mathcal{G}^* | P) \\ &= \arg \min_{P, D} \underbrace{-\log p(I | \mathcal{G}^*, D, P)}_{\text{BA Term}} \underbrace{-\log p(D | \mathcal{G}^*, P)}_{\text{Depth Regularization Term}} - \log p(\mathcal{G}^* | P) \end{aligned} \quad (8)$$

- **BA Term:** 기존 DSO의 photometric loss (E_{photo}) 그대로 사용
 - $\mathcal{G}^*$ 고정 시 $-\log p(\mathcal{G}^* | P)$ 는 P 에 대한 영향 미미 $\rightarrow$ 무시
- **Depth Regularization Term:** E-step의 $\mathcal{G}^*$ 에서 렌더링한 depth와 DSO depth의 **weighted average**
 - 매 iteration마다 depth re-rendering은 연산 과다 $\rightarrow$ averaging으로 대체
 - Basin of attraction 내에서 안정적인 초기화 역할 $\rightarrow$ high-frequency geometric detail 복원 유리

Method

Joint Optimization: EM에서 Loss 유도 정리

PRML EM 일반 형태:

1. E-step: $q(\mathbf{Z}) = p(\mathbf{Z}|\mathbf{X}, \boldsymbol{\theta}^{\text{old}})$
2. M-step: $\boldsymbol{\theta}^{\text{new}} = \arg \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}})$

δ -function 근사 시:

1. E-step: $\mathbf{Z}^* = \arg \max_{\mathbf{Z}} p(\mathbf{Z}|\mathbf{X}, \boldsymbol{\theta})$
2. M-step: $\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \log p(\mathbf{X}, \mathbf{Z}^*|\boldsymbol{\theta})$

GSO-SLAM에서:

1. E-step (P, D 고정, $\mathcal{G}$ 최적화):

- $\mathcal{G}^* = \arg \min_{\mathcal{G}} \mathcal{L}_{\text{E-step}}$ (Eq. 7)
- RGB loss + Depth loss + Normal loss

2. M-step ($\mathcal{G}^*$ 고정, P, D 최적화):

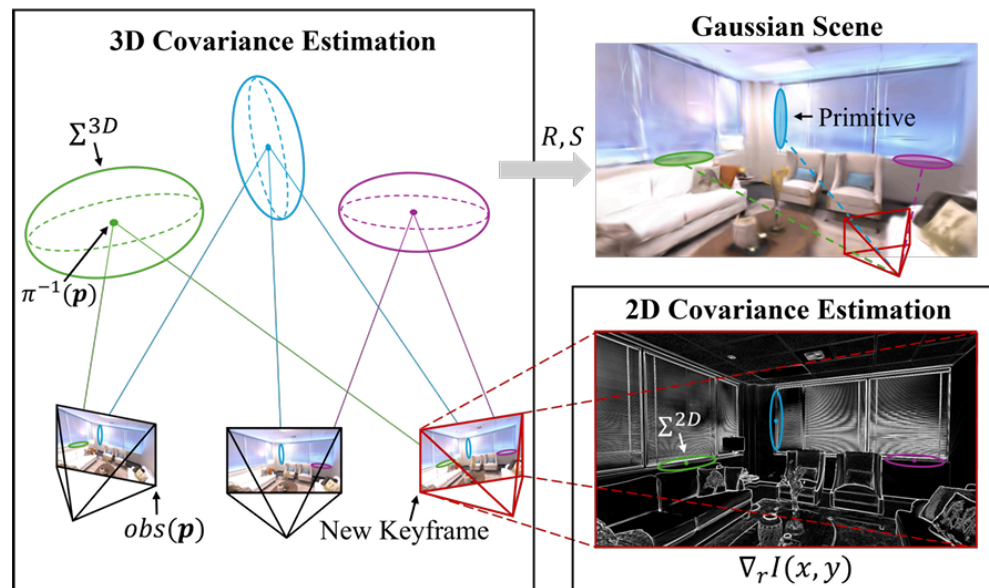
- $(P^*, D^*) = \arg \min_{P, D} \mathcal{L}_{\text{M-step}}$ (Eq. 8)
- BA Term + Depth Regularization

3. 반복 $\rightarrow$ 수렴 보장 (ELBO 단조 증가)

Method

Gaussian Splat Initialization:

- DSO는 최적화 과정에서 **image gradient**를 이미 계산
- Image gradient를 projected Gaussian Splat distribution의 확률적 추정으로 사용
- 추가 연산 없이 초기 Gaussian 파라미터 결정
- 3단계 프로세스:
 1. Keyframe의 image intensity와 gradient로부터 **2D covariance** 추정
 2. Multiple keyframe의 2D covariance를 결합하여 **3D covariance** 계산
 3. Eigen-decomposition으로 rotation과 scaling 파라미터 추출



Method

Gaussian Splat Initialization: 2D Covariance 추정

- Image intensity가 Gaussian distribution을 따른다고 가정:

$$I(x, y) = \frac{1}{2\pi|\Sigma_{2D}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{r} - \mathbf{p})^\top (\Sigma_{2D})^{-1}(\mathbf{r} - \mathbf{p})\right) \quad (9)$$

- $\mathbf{r} = [x, y]^\top$: image 좌표
- $\mathbf{p} = [p_x, p_y]^\top$: distribution의 center
- Log-probability의 gradient를 계산:

$$\nabla_{\mathbf{r}} \log I(x, y) = -\alpha(\Sigma_{2D})^{-1}(\mathbf{r} - \mathbf{p}) \quad (10)$$

- α : scaling factor
- 다양한 $\mathbf{r}$ 에 대해 이 식을 세우고, **least-squares**로 Σ_{2D} 를 추정

Method

Gaussian Splat Initialization: 2D $\rightarrow$ 3D Covariance

- 동일한 3D point가 여러 keyframe $\text{obs}(\mathbf{p})$ 에서 관측됨
- 각 keyframe i 에서의 2D covariance Σ_i^{2D} 는 하나의 3D covariance Σ_{3D} 에 대응
- 2D-3D covariance 관계 (EWA Splatting):

$$\Sigma_i^{2D} = J_i W_i \Sigma_{3D} W_i^\top J_i^\top, \quad i \in \text{obs}(\mathbf{p}) \quad (11)$$

- J : 3D point $\rightarrow$ image plane의 Jacobian matrix
- W : world $\rightarrow$ camera 좌표 변환
- 모든 observation의 방정식을 선형화하여 **least-squares**로 Σ_{3D} 추정

Method

Gaussian Splat Initialization: Covariance 보정

- Closed-form 해로 얻은 covariance가 유효한 covariance matrix의 조건을 만족하지 못할 수 있음
 - Covariance matrix는 **symmetric positive semi-definite**이어야 함
- 보정 전략:

$$\tilde{\Sigma}_{3D} = \begin{cases} \Sigma_{3D} + \epsilon \mathbf{I} & \text{(Regularization)} \\ U\tilde{\Lambda}U^\top & \text{(Eigenvalue clipping)} \end{cases} \quad (12)$$

- $\epsilon > 0$: 작은 상수
- U : Σ_{3D} 의 eigenvector matrix
- $\tilde{\Lambda} = \text{diag}(\max(\lambda_i, \epsilon))$: 음의 eigenvalue를 clip
- Eigen-decomposition으로 최종 파라미터 추출:
 - Eigenvector matrix → **Rotation matrix** R
 - Eigenvalue의 대각 행렬 → **Scaling matrix** S (최소 eigenvalue를 0으로 설정 → 2DGS)
 - α : S 의 최대 원소로 나눈 preset 값으로 정의하여 regularize

Method

SLAM Framework: Localization & M-step

1. 새 **frame** 도착 시: Eq. (2)의 energy function으로 two-frame direct image alignment → pose 추정
2. **Keyframe** 선택: FOV 변화, translation, exposure time 기반
 - 새 keyframe이 선택되면 sliding window에 추가
3. **Local BA (M-step):**
 - Camera pose, affine brightness parameter, inverse depth, camera intrinsic 최적화
 - 이것이 EM의 **M-step**에 해당
4. **Depth Regularization:**
 - Keyframe의 sparse depth map + 이전 E-step에서 업데이트된 2DGS rendered depth map
 - 두 depth의 **평균**을 초기 depth estimate로 사용
 - Dense reconstruction의 이점을 활용하면서 DSO-derived depth의 신뢰성 보존
 - 이후 Eq. (3)으로 camera pose와 semi-dense depth map을 최적화

Method

SLAM Framework: E-step Optimization

- Main thread가 tracking과 keyframe 선택을 처리하는 동안, **2DGS scene optimization**은 병렬 **thread**에서 실행
- 새 keyframe이 선택될 때마다:
 1. **Gaussian Splat Initialization** (Sec. III-C)으로 새 Gaussian splat의 초기 형태 정의
 2. Reconstructed scene에 삽입
 3. 원본 GS의 **densification** 기법도 추가로 적용하여 finer detail 개선
- **Windowed refinement:**
 - 최근 추가된 keyframe을 우선적으로 최적화
 - Active window가 충분히 학습되면, 추가 keyframe을 샘플링하여 다양한 viewpoint 반영
- **Real-time 성능 보장:**
 - E-step이 별도 thread에서 지속적으로 scene Gaussian을 refine
 - Front-end는 real-time으로 유지

Experiments

TABLE II: **Camera Tracking Accuracy on TUM-RGBD Dataset for Monocular and RGB-D (ATE RMSE [cm] ↓).** We categorize the evaluated systems based on their tracking approach into direct and feature-based methods.

Method	Base	fr1/desk	fr2/xyz	fr3/office	Avg.
<i>RGB-D Methods</i>					
Point-SLAM [7]	Direct	2.73	1.30	3.52	2.52
SplaTAM [12]	Direct	3.30	1.32	5.12	3.25
MonoGS [10]	Direct	1.47	1.39	1.66	1.02
<i>Monocular Methods</i>					
Orbeez-SLAM [4]	Feature	1.23	0.23	1.71	1.05
Photo-SLAM [11]	Feature	1.64	0.25	1.16	1.34
MonoGS [10]	Direct	3.29	4.22	3.70	3.73
MGSO* [15]	Direct	3.91	0.21	10.12	4.74
Ours	Direct	2.54	0.20	6.47	3.07

TABLE II: **Camera Tracking Accuracy on TUM-RGBD Dataset for Monocular and RGB-D (ATE RMSE [cm] ↓).** We categorize the evaluated systems based on their tracking approach into direct and feature-based methods.

Method	Base	fr1/desk	fr2/xyz	fr3/office	Avg.
<i>RGB-D Methods</i>					
Point-SLAM [7]	Direct	2.73	1.30	3.52	2.52
SplaTAM [12]	Direct	3.30	1.32	5.12	3.25
MonoGS [10]	Direct	1.47	1.39	1.66	1.02
<i>Monocular Methods</i>					
Orbeez-SLAM [4]	Feature	1.23	0.23	1.71	1.05
Photo-SLAM [11]	Feature	1.64	0.25	1.16	1.34
MonoGS [10]	Direct	3.29	4.22	3.70	3.73
MGSO* [15]	Direct	3.91	0.21	10.12	4.74
Ours	Direct	2.54	0.20	6.47	3.07

Experiments

TABLE III: **Evaluation Results of Map Quality and System FPS on Replica Dataset.** Results denote averages across eight sequences. Our method achieves real-time performance yielding the highest map quality in most scenes. The results for MonoGS correspond to its sequential implementation.

Method	PSNR [dB] $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Depth L1 [cm] $\downarrow$	FPS [Hz] $\uparrow$
<i>RGB-D Methods</i>					
Point-SLAM [7]	35.55	0.976	0.118	-	0.49
SplaTAM [12]	34.13	0.970	0.099	-	0.21
<i>Monocular Methods</i>					
Photo-SLAM [11]	30.91	0.911	0.099	19.37	30
MonoGS [10]	32.01	0.919	0.171	25.36	0.84
MGSO* [15]	28.17	0.855	0.204	11.54	30
Ours	34.48	0.943	0.060	8.12	30

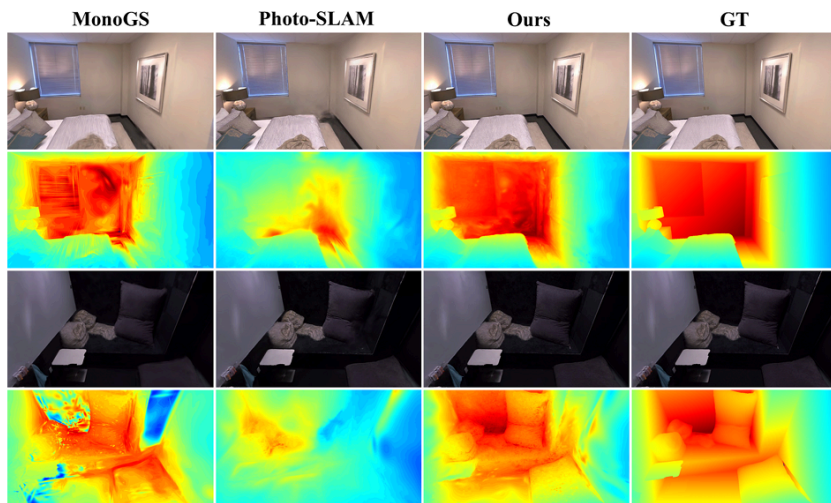


TABLE IV: **Evaluation Results of Map Quality and System FPS on TUM-RGBD Dataset.** The reported values represent the average across three sequences.

Method	PSNR [dB] $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS [Hz] $\uparrow$
<i>RGB-D Methods</i>				
Point-SLAM [7]	21.30	0.731	0.453	0.39
SplaTAM [12]	23.50	0.909	0.156	0.41
<i>Monocular Methods</i>				
Photo-SLAM [11]	19.78	0.701	0.232	30
MonoGS [10]	22.15	0.741	0.321	3.11
MGSO* [15]	19.84	0.690	0.331	30
Ours	20.52	0.727	0.263	30



Experiments

Real-world & Scalability

Self-captured Dataset (Quadrupedal Robot):

- 상당한 camera motion jitter에도 불구하고 우수한 map reconstruction quality



Large-scale INS Dataset:

TABLE V: **Tracking Accuracy Evaluation on the Large-scale INS Dataset.** We report the Absolute Trajectory Error RMSE [m] on the Indoor Long Corridor (LC) sequences.

Method	Loop Closure	LC_2	LC_3	LC_4	Avg.
MonoGS [10]	✗	13.19	10.65	3.27	9.48
Photo-SLAM [11]	✗	2.75	14.68	3.92	6.18
Photo-SLAM [11]	✓	0.21	8.71	3.17	3.12
Ours	✗	0.47	0.44	1.00	0.64

- Pure VO로 MonoGS보다 높은 정확도
- Photo-SLAM (Loop Closure 있음)보다 LC_3, LC_4에서 우수
- **Loop Closing 없이도** 누적 drift 최소화
- PSNR: 23.11 dB vs MonoGS 16.76 / Photo-SLAM 17.62

Experiments

Runtime Analysis

TABLE VI: **System Runtime Analysis across Different Scales.** We report the average runtime per module (in ms) on the small-scale (fr3-office) and large-scale (LC_2) datasets.

Sequence	Scale	DSO Tracking	EM Opt (M-step)	GS Init	Render (E-step)
fr3-office	Small	1.37 ± 2.47	50.59 ± 9.40	5.33 ± 1.64	1.67 ± 0.28
LC_2	Large	0.80 ± 0.64	44.79 ± 8.79	5.27 ± 1.10	1.87 ± 0.92

- Tracking: 항상 low latency (local keyframe 기반)
- M-step: windowed optimization → scene scale에 무관하게 일정한 복잡도
- 카메라 모션이 runtime에 더 큰 영향 (handheld > smooth trajectory)
- GPU memory: Small 2.8GB / Large 5.8GB

Experiments

Ablation Study: Gaussian Splat Initialization

TABLE VII: **Ablation Study of Gaussian Splat Initialization.** Reported values represent the average across 8 scenes of Replica dataset.

Method	PSNR [dB] $\uparrow$	Additional	
		Iterations	Time [s]
KNN	25.51	2700	64.4
Const.	31.68	1200	20.4
Obs. (Proposed)	34.48	-	-
Pre.	33.87	-	-
Int.	33.92	-	-

- **KNN 기반:** 기존 GS/Photo-SLAM 방식, PSNR 25.51, 추가 2700 iter (64.4s) 필요
- **Constant (Isotropic):** PSNR 31.68, 추가 1200 iter (20.4s) 필요

Experiments

Ablation Study: Joint Optimization

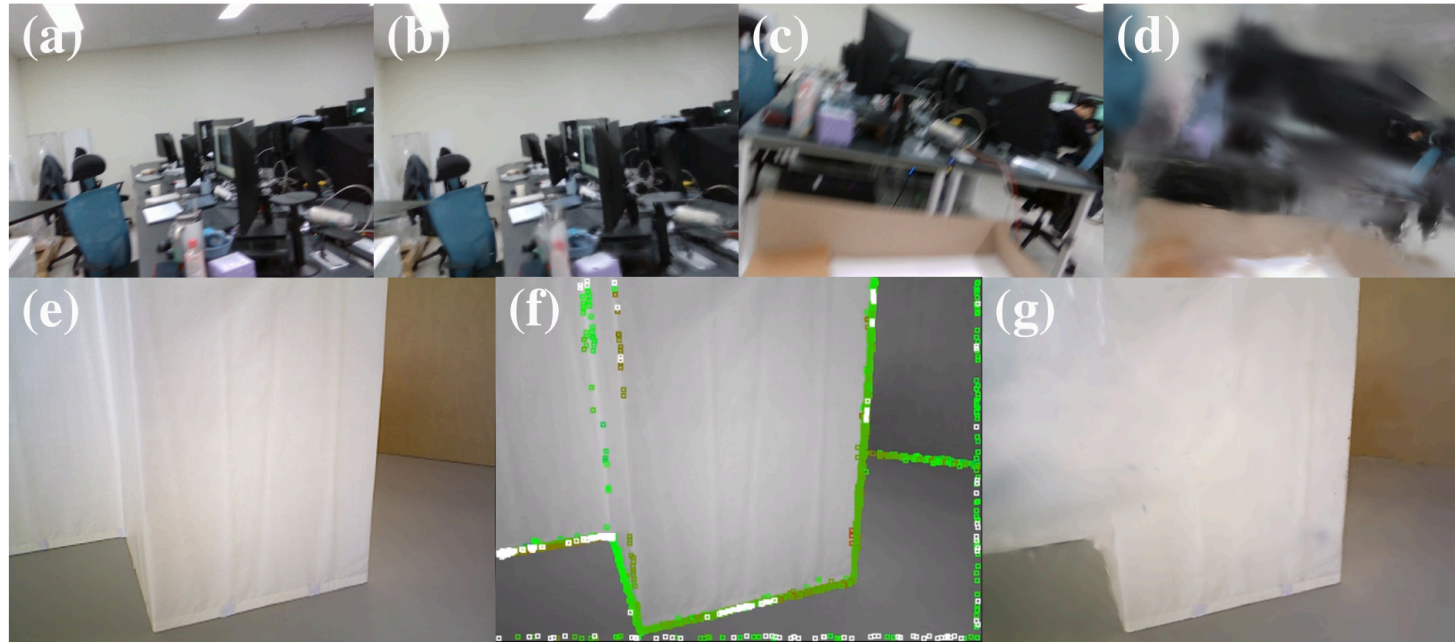
TABLE VIII: **Ablation of Joint Optimization.** We report the average results across all scenes of Replica dataset. Our results show that the joint optimization significantly improves both the geometric fidelity and tracking accuracy.

Joint Opt.	PSNR [dB] $\uparrow$	ATE RMSE [cm] $\downarrow$	Depth L1 [cm] $\downarrow$
$\times$	34.10	0.688	9.744
$\checkmark$	34.48	0.462	8.124

- Joint Optimization 없이 (naive coupling): PSNR 34.10, ATE 0.688, Depth L1 9.744
- Joint Optimization 적용: PSNR **34.48**, ATE **0.462**, Depth L1 **8.124**
- Scene geometry가 제대로 공유되면 tracking과 mapping 모두 개선

Experiments

Failure Cases



- **(a, b):** 안정적인 motion → robust
- **(c, d):** 심한 motion blur → photometric consistency 위반 → tracking drift → geometric distortions
- **(e, f, g):** Textureless region (흰 벽) → image gradient 부족 → sampling 불가 → 불완전한 geometry
 - 그럼에도 제안된 initialization은 PSNR 25.51로 Constant (22.27), KNN (25.23) 대비 우수

Conclusion

1. **Bidirectional Coupling:** VO와 GS를 seamless하게 통합

- 추가 연산 비용 없이 tracking과 mapping 성능 모두 향상

2. **EM Framework:** Joint optimization을 EM으로 정식화

- E-step: $\mathcal{G}$ 를 MAP로 추정 (RGB + Depth + Normal loss)
- M-step: (P, D) 를 BA + Depth Regularization으로 최적화

3. **Gaussian Splat Initialization:** DSO의 image gradient 재활용

- 2D covariance → 3D covariance 추정 → clipping으로 보정
- 기존방식 초기화 대비 수렴 가속화 및 reconstruction quality 향상

4. **성능:** Real-time (30 FPS)으로 SOTA 수준의 photometric/geometric fidelity 및 tracking accuracy 달성